Congress Receipts: how one video gets uploaded

YouTube API Services audit evidence. Channel @CongressReceipts (Congress Receipts). Site https://congressreceipts.pages.dev. Code: github.com/houstoninvestments-jpg/congress-receipts

1. Scheduled job starts
GitHub Actions, up to 3 slots a day. Unsure about a trade means skip.

2. Pick and check one trade
From a public House filing. Code checks plus two independent reviews.

3. Make the video
Kokoro AI voice, Remotion render, under 40 seconds, 1080x1920.

4. Sign in
Refresh token from the one-time owner authorization; short-lived access token.

5. Upload
Resumable upload of our own video. First-ever upload is private. containsSyntheticMedia = true.
`youtube.videos.insert`

6. Lock test (first upload only)
Ask for unlisted, read status back; private means locked. Then thumbnail.
`youtube.videos.update · youtube.videos.list · youtube.thumbnails.set`

Run record: Receipt No. 001 (the first upload)

Run	GitHub Actions, Daily Receipt workflow, scheduled run 40225070983, 2026-10-25T19:18:55.139Z
Video	GLCF2kvhLUw (uploaded as private: the first upload is always private)
Title	Receipt No. 001: Rep. Jonathan L. Jackson bought Estee Lauder Companies
Status sent	privacyStatus=private, selfDeclaredMadeForKids=false, containsSyntheticMedia=true
Lock test result	status stayed private after asking for unlisted → locked: yes. The video stays private; nothing was made public.
Source filing	https://disclosures-clerk.house.gov/public_disc/ptr-pdfs/2026/20035551.pdf

Run notes from the job log (secrets are never printed; GitHub masks them):

```
Receipt No. 001 upload run: GitHub Actions, Daily Receipt workflow, scheduled run 40225070983 (2026-10-25).
Lines from the job log. Key names only; secret values are never printed.

note: keys set (names only): ANTHROPIC_API_KEY MARKETSTACK_API_KEY YOUTUBE_CLIENT_ID YOUTUBE_CLIENT_SECRET CLOUDFL
ARE_API_TOKEN CLOUDFLARE_ACCOUNT_ID
note: website: cloudflare https://congressreceipts.pages.dev
note: authorize link ready at https://congressreceipts.pages.dev/auth/youtube
note: YouTube connected: yes, Congress Receipts
[2026-10-25T19:16:51.524Z] ok   data: 439 House members; 437 reports in the Clerk's index, 60 new parsed
[2026-10-25T19:16:51.536Z] ok   pick: 418 purchases qualify, 386 after the party-balance and repeat rules; set asi
de: {"filing scanned":49,"not a listed stock with a ticker":116,"not a purchase":685,"thinly traded foreign share
(over the counter)":4,"filer not matched to one sitting member":7}
[2026-10-25T19:17:05.894Z] FAIL check: Jonathan L. Jackson / DDOG: problems: Minor wording issue: the description
says 'reported a joint purchase ... on September 28, 2026', which could be read as the report date. The next sente
nce gives the correct filing date (October 19, 2026), so this is not a factual mismatch. Rewording to 'a joint pur
chase ... made September 28, 2026' would be clearer.
[2026-10-25T19:17:37.842Z] ok   check: Jonathan L. Jackson / EL: code checks and both reviews passed
[2026-10-25T19:17:51.560Z] ok   voice: AI voice (Kokoro open model), 36.4s
[2026-10-25T19:18:42.486Z] ok   render: work/2026-10-25/receipt-001.mp4 (1080x1920, 36.37s)
[2026-10-25T19:18:55.139Z] first upload lock test: status stayed private after asking for unlisted
[2026-10-25T19:18:55.140Z] FAIL publish: status stayed private after asking for unlisted
[2026-10-25T19:18:55.141Z] result: rendered work/2026-10-25/receipt-001.mp4
note: rendered | first upload is locked private; audit answers prepared in docs/youtube-audit-request.md
note: Rep. Jonathan L. Jackson (D-IL) bought EL $50,001 - $100,000, traded 2026-09-28, filed 2026-10-19, 0 days la
te | https://disclosures-clerk.house.gov/public_disc/ptr-pdfs/2026/20035551.pdf
```

```
note: EL 13.7% vs SPY -0.7% (2026-09-28 to 2026-10-23), stamp BEAT THE MARKET
note: filing card untouched (0.00% changed), result untouched by the stamp (0.00%)
note: Receipt No. 001: Rep. Jonathan L. Jackson bought Estee Lauder Companies
note: GLCF2kvhLUw, private, LOCKED private by Google;
note: $0.2609 over 7 model calls (writer claude-fable-5-1 1593+679 tok; checker-a claude-opus-5-5 1302+328 tok; ch
ecker-b claude-opus-5-5 1266+500 tok; writer claude-fable-5-1 1619+1654 tok; writer-retry claude-fable-5-1 1953+89
7 tok; checker-a claude-opus-5-5 1331+220 tok; checker-b claude-opus-5-5 1295+299 tok)
note: YouTube channel: Congress Receipts @congressreceipts
note: https://congressreceipts.pages.dev answered 200 for: / /receipts/ /scoreboard/ /how/ /privacy/ /terms/
note: home and scoreboard labeled and disclosed; 0 published receipt pages carry their chart link; TradingView lin
ks open
note: already connected; the link says so
```

Description sent with the upload:

```
Rep. Jonathan L. Jackson (D-IL) reported a joint purchase of Estee Lauder Companies (EL) worth $50,001 - $100,000
on September 28, 2026. The report was filed October 19, 2026.

Amount is the range on the filing ($50,001 - $100,000), not an exact figure.
Prices: closing prices September 28, 2026 to October 23, 2026, from Marketstack end-of-day prices. S&P 500 shown a
s SPY, price change only.

Source filing (U.S. House Clerk): https://disclosures-clerk.house.gov/public_disc/ptr-pdfs/2026/20035551.pdf
The receipt and the math: https://congressreceipts.pages.dev/receipt/001-jonathan-l-jackson-el/

Full EL chart on TradingView (paid link): https://www.tradingview.com/symbols/NYSE-EL/?aff_id=1173299&aff_sub=rece
ipt-001&source=youtube
Paid link: TradingView pays us if you sign up for a paid plan through it. It has no effect on which trades we sho
w.

Not financial advice. Facts from public filings.
Disclosure: the narration is an AI voice (Kokoro open model). The filing card is a clean recreation of the officia
l report.

#CongressReceipts #STOCKAct #Congress #Shorts
```

Upload script, unchanged: src/publish/youtube.ts

```
import { readFileSync, statSync } from "node:fs";
import type { Ledger, Receipt } from "../types.js";
import { config } from "../lib/config.js";
import { log, path, requireEnv } from "../lib/util.js";
import { refreshToken } from "../auth/store.js";
import { writeAuditRequest } from "./audit.js";
import { longDate } from "../receipt/facts.js";
import { DESCRIPTION_DISCLOSURE, chartLink } from "../site/affiliate.js";

export interface Uploader {
  token(): Promise<string>;
  fetch: typeof fetch;
}

/** OAuth refresh-token client. The token comes from the one-link authorization (or a hand-set secret). */
export function oauthUploader(fetchImpl: typeof fetch = fetch): Uploader {
  let cached: { token: string; until: number } | undefined;
  return {
    fetch: fetchImpl,
    async token() {
      if (cached && Date.now() < cached.until) return cached.token;
      const rt = await refreshToken();
      if (!rt) throw new Error("YouTube is not connected yet; open the authorize link in docs/SETUP.md");
      const res = await fetchImpl("https://oauth2.googleapis.com/token", {
        method: "POST",
        headers: { "content-type": "application/x-www-form-urlencoded" },
        body: new URLSearchParams({ grant_type: "refresh_token", refresh_token: rt, client_id: requireEnv("YOUTUBE
_CLIENT_ID"), client_secret: requireEnv("YOUTUBE_CLIENT_SECRET") }),
      });
      if (!res.ok) throw new Error(`YouTube sign-in failed (${res.status}); the one-link authorization needs redoi
ng`);
      const j = (await res.json()) as { access_token: string; expires_in: number };
      cached = { token: j.access_token, until: Date.now() + (j.expires_in - 120) * 1000 };
      return j.access_token;
    },
  };
}

export const HASHTAGS = ["#CongressReceipts", "#STOCKAct", "#Congress", "#Shorts"];

export function receiptUrl(r: Receipt) {
  return `${config().site.baseUrl}/receipt/${r.slug}/`;
}

export function videoDescription(r: Receipt, voiceLabel: string) {
  const c = config();
  const f = r.facts;
  return [
    r.script.description.replace(/#/g, ""),
    "",
    `Amount is the range on the filing (${f.amountText}), not an exact figure.`,
    f.prices ? `Prices: closing prices ${longDate(f.prices.stock.start.date)} to ${longDate(f.prices.stock.end.dat
e)}, from ${f.prices.source.name}. S&P 500 shown as SPY, price change only.` : "",
    "",
    `Source filing (U.S. House Clerk): ${f.pdfUrl}`,
    `The receipt and the math: ${receiptUrl(r)}`,
    ...(c.affiliate.enabled ? ["", `Full ${f.ticker} chart on TradingView (paid link): ${chartLink(f.ticker, r.num
ber, "youtube")}`, DESCRIPTION_DISCLOSURE] : []),
    "",
    c.channel.notAdvice,
    `Disclosure: the narration is an ${voiceLabel}. The filing card is a clean recreation of the official report.
`,
    "",
    HASHTAGS.join(" "),
  ]
    .filter((l, i, a) => !(l === "" && a[i - 1] === ""))
    .join("\n");
}

/** The exact metadata sent to YouTube. Saved on every run, so dry runs show what would be uploaded. */
```

```
export function uploadRequest(r: Receipt, voiceLabel: string, privacy?: "public" | "private" | "unlisted") {
  const cfg = config();
  return {
    snippet: {
      title: r.script.title,
      description: videoDescription(r, voiceLabel),
      tags: ["congress stock trades", "STOCK Act", "periodic transaction report", "congress receipts", r.facts.tic
ker, r.facts.company].filter((t) => t.length < 60),
      categoryId: cfg.youtube.categoryId,
      defaultLanguage: cfg.youtube.defaultLanguage,
      defaultAudioLanguage: cfg.youtube.defaultLanguage,
    },
    status: {
      privacyStatus: privacy ?? cfg.youtube.privacyStatus,
      selfDeclaredMadeForKids: false,
      // YouTube's altered-or-synthetic content flag. Set on every upload for the AI voice.
      containsSyntheticMedia: true,
    },
  };
}

const API = "https://www.googleapis.com/youtube/v3";

async function api(u: Uploader, method: string, url: string, body?: unknown) {
  const res = await u.fetch(url, { method, headers: { authorization: `Bearer ${await u.token()}`, "content-type":
"application/json" }, body: body ? JSON.stringify(body) : undefined });
  const text = await res.text();
  if (!res.ok) throw new Error(`YouTube ${method} ${url.split("?")[0]} -> ${res.status}: ${text.slice(0, 300)}`);
  return text ? JSON.parse(text) : {};
}

export async function uploadVideo(u: Uploader, file: string, meta: ReturnType<typeof uploadRequest>): Promise<stri
ng> {
  const size = statSync(file).size;
  const init = await u.fetch(`https://www.googleapis.com/upload/youtube/v3/videos?uploadType=resumable&part=snippe
t,status`, {
    method: "POST",
    headers: { authorization: `Bearer ${await u.token()}`, "content-type": "application/json; charset=UTF-8", "x-u
pload-content-length": String(size), "x-upload-content-type": "video/mp4" },
    body: JSON.stringify(meta),
  });
  if (!init.ok) throw new Error(`upload start failed ${init.status}: ${(await init.text()).slice(0, 300)}`);
  const session = init.headers.get("location");
  if (!session) throw new Error("upload start returned no session URL");
  const put = await u.fetch(session, { method: "PUT", headers: { "content-type": "video/mp4", "content-length": St
ring(size) }, body: readFileSync(file) });
  const text = await put.text();
  if (!put.ok) throw new Error(`upload failed ${put.status}: ${text.slice(0, 300)}`);
  return (JSON.parse(text) as { id: string }).id;
}

export async function videoStatus(u: Uploader, id: string) {
  const j = await api(u, "GET", `${API}/videos?part=status&id=${id}`);
  return j.items?.[0]?.status as { privacyStatus: string; uploadStatus: string } | undefined;
}

export async function setPrivacy(u: Uploader, id: string, privacyStatus: string) {
  return api(u, "PUT", `${API}/videos?part=status`, { id, status: { privacyStatus, selfDeclaredMadeForKids: false,
containsSyntheticMedia: true } });
}

/** Is this API project locked to private uploads? Test it: try the private test video as unlisted (never public).
*/
export async function lockTest(u: Uploader, id: string): Promise<{ locked: boolean; detail: string }> {
  try {
    await setPrivacy(u, id, "unlisted");
  } catch (e) {
    return { locked: true, detail: `changing privacy was refused: ${String(e).slice(0, 200)}` };
  }
  const st = await videoStatus(u, id);
  if (st?.privacyStatus === "unlisted") return { locked: false, detail: "private test video could be made unliste
d; uploads are not locked" };
  return { locked: true, detail: `status stayed ${st?.privacyStatus ?? "unknown"} after asking for unlisted` };
}
```

```
export async function channelInfo(u: Uploader): Promise<{ id: string; title: string; handle: string }> {
  const j = await api(u, "GET", `${API}/channels?part=snippet&mine=true`);
  const c = j.items?.[0];
  if (!c) throw new Error("no channel found for this sign-in");
  return { id: c.id, title: c.snippet?.title ?? "", handle: c.snippet?.customUrl ?? "" };
}

export async function setThumbnail(u: Uploader, videoId: string, jpeg: string): Promise<{ ok: boolean; detail: str
ing }> {
  try {
    const body = readFileSync(jpeg);
    const res = await u.fetch(`https://www.googleapis.com/upload/youtube/v3/thumbnails/set?videoId=${encodeURIComp
onent(videoId)}&uploadType=media`, {
      method: "POST",
      headers: { authorization: `Bearer ${await u.token()}`, "content-type": "image/jpeg", "content-length": Strin
g(body.length) },
      body,
    });
    const text = await res.text();
    if (!res.ok) return { ok: false, detail: `thumbnail not set (${res.status}): ${text.replace(/\s+/g, " ").slice
(0, 200)}` };
    return { ok: true, detail: "thumbnail set from the cover frame" };
  } catch (e) {
    return { ok: false, detail: `thumbnail not set: ${String(e).slice(0, 200)}` };
  }
}

/** While uploads are locked: re-test once a day. If the lock lifted, put the test video back to private and carry
on. */
export async function recheckLock(u: Uploader, ledger: Ledger): Promise<{ locked: boolean; detail: string }> {
  const check = ledger.firstUploadCheck;
  if (!check?.locked) return { locked: false, detail: "not locked" };
  const res = await lockTest(u, check.videoId);
  if (!res.locked) {
    await setPrivacy(u, check.videoId, config().youtube.firstUploadPrivacy);
    ledger.firstUploadCheck = { ...check, locked: false, checkedAt: new Date().toISOString(), detail: `lock lifte
d: ${res.detail}` };
  } else ledger.firstUploadCheck = { ...check, checkedAt: new Date().toISOString(), detail: res.detail };
  return res;
}

export interface PublishResult {
  videoId: string;
  lockedPrivate: boolean;
  privacy: "public" | "unlisted" | "private";
  detail: string;
  thumbnail?: { ok: boolean; detail: string };
}

export async function publishShort(u: Uploader, r: Receipt, file: string, voiceLabel: string, ledger: Ledger, cove
rJpg?: string): Promise<PublishResult> {
  if (ledger.firstUploadCheck?.locked) throw new Error("uploads are locked private until the YouTube API audit pas
ses; see docs/youtube-audit-request.md");
  let res: PublishResult;
  if (!ledger.firstUploadCheck) {
    // The very first upload goes up private and tests the lock before anything is public.
    const id = await uploadVideo(u, file, uploadRequest(r, voiceLabel, "private"));
    const lock = await lockTest(u, id);
    ledger.firstUploadCheck = { videoId: id, checkedAt: new Date().toISOString(), ...lock };
    log(`first upload lock test: ${lock.detail}`);
    if (lock.locked) {
      writeAuditRequest(path("docs/youtube-audit-request.md"), lock.detail);
      return { videoId: id, lockedPrivate: true, privacy: "private", detail: lock.detail };
    }
    const keep = config().youtube.firstUploadPrivacy;
    await setPrivacy(u, id, keep);
    res = { videoId: id, lockedPrivate: false, privacy: keep, detail: `${lock.detail}; test video kept ${keep}` };
  } else {
    const privacy = config().youtube.privacyStatus;
    const id = await uploadVideo(u, file, uploadRequest(r, voiceLabel));
    res = { videoId: id, lockedPrivate: false, privacy, detail: `uploaded ${id} as ${privacy}` };
  }
  if (coverJpg) res.thumbnail = await setThumbnail(u, res.videoId, coverJpg);
```

```
  return res;
}
```

Daily workflow, unchanged: .github/workflows/daily.yml

```
name: Daily Receipt

on:
  schedule:
    # Three slots a day, a ceiling not a quota: each posts only if a trade passes every check.
    # About 9:37am, 2:37pm and 7:37pm Pacific (an hour earlier in winter; GitHub cron runs on UTC).
    # Scam or Not runs at 14:23 UTC, so these never overlap it.
    - cron: "37 16 * * *"
    - cron: "37 21 * * *"
    - cron: "37 2 * * *"
  workflow_dispatch:
    inputs:
      dry_run:
        description: "Dry run (render, check, build site; upload nothing)"
        type: boolean
        default: false
      keep_sample:
        description: "Keep this run's video on a hidden ref (refs/previews/receipt-sample) so it can be fetched"
        type: boolean
        default: false
      reset_youtube:
        description: "Forget the connected YouTube channel so the authorize link works again"
        type: boolean
        default: false

permissions:
  contents: write # save the ledger and parsed filings back to main; publish gh-pages until Cloudflare is set
  issues: write # health check opens an issue only when something breaks

concurrency:
  group: daily-receipt
  cancel-in-progress: false

jobs:
  receipt:
    runs-on: ubuntu-latest
    timeout-minutes: 60
    env:
      ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
      MARKETSTACK_API_KEY: ${{ secrets.MARKETSTACK_API_KEY }}
      YOUTUBE_CLIENT_ID: ${{ secrets.YOUTUBE_CLIENT_ID }}
      YOUTUBE_CLIENT_SECRET: ${{ secrets.YOUTUBE_CLIENT_SECRET }}
      CLOUDFLARE_API_TOKEN: ${{ secrets.CLOUDFLARE_API_TOKEN }}
      CLOUDFLARE_ACCOUNT_ID: ${{ secrets.CLOUDFLARE_ACCOUNT_ID }}
      KOKORO_DIR: ${{ github.workspace }}/models/kokoro
      # Optional contact address for the privacy and terms pages. Never committed to this repo.
      CONTACT_EMAIL: ${{ vars.CONTACT_EMAIL || secrets.CONTACT_EMAIL }}
      GH_TOKEN: ${{ github.token }}
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: requirements-voice.txt

      - name: Install
        run: |
          sudo apt-get update -q && sudo apt-get install -y -q ffmpeg poppler-utils
          npm ci
          pip install -q -r requirements-voice.txt

      - name: Voice model (Kokoro)
        uses: actions/cache@v4
```

```
        with:
          path: models/kokoro
          key: kokoro-v1.0-int8

      - name: Download voice model if not cached
        run: |
          mkdir -p models/kokoro && cd models/kokoro
          [ -f kokoro-v1.0.int8.onnx ] || curl -sSL -o kokoro-v1.0.int8.onnx https://github.com/thewh1teagle/kokor
o-onnx/releases/download/model-files-v1.0/kokoro-v1.0.int8.onnx
          [ -f voices-v1.0.bin ] || curl -sSL -o voices-v1.0.bin https://github.com/thewh1teagle/kokoro-onnx/relea
ses/download/model-files-v1.0/voices-v1.0.bin

      - name: Check which keys are set (names only)
        run: |
          set_list=""; missing=""
          for n in ANTHROPIC_API_KEY MARKETSTACK_API_KEY YOUTUBE_CLIENT_ID YOUTUBE_CLIENT_SECRET CLOUDFLARE_API_TO
KEN CLOUDFLARE_ACCOUNT_ID; do
            if [ -n "${!n}" ]; then set_list="$set_list $n"; else missing="$missing $n"; fi
          done
          echo "::notice title=Keys set::${set_list:- none}"
          [ -z "$missing" ] || echo "::notice title=Keys not set yet::$missing"

      - name: Pick the website address
        id: site
        run: npx tsx src/cli.ts site-target

      - name: Forget the YouTube connection
        if: inputs.reset_youtube && steps.site.outputs.kind == 'cloudflare'
        run: npx tsx src/cli.ts auth-reset

      - name: Ready the one-link YouTube authorize page
        id: auth
        if: steps.site.outputs.kind == 'cloudflare'
        continue-on-error: true
        run: npx tsx src/cli.ts auth-arm

      - name: Make the Receipt
        id: run
        run: |
          # Live only when every piece exists: models, a licensed price source, and a connected channel.
          if [ "${{ inputs.dry_run }}" = "true" ] || [ -z "$ANTHROPIC_API_KEY" ] || [ -z "$MARKETSTACK_API_KEY" ]
|| [ "${{ steps.auth.outputs.youtube }}" != "connected" ]; then
            echo "mode=dry" >> "$GITHUB_OUTPUT"
            npx tsx src/cli.ts daily --dry-run
          else
            echo "mode=live" >> "$GITHUB_OUTPUT"
            npx tsx src/cli.ts daily
          fi

      - name: Keep the video for 14 days
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: receipt-${{ github.run_id }}
          path: |
            work/**/*.mp4
            work/**/*-cover.jpg
            work/**/upload-request.json
            work/**/run-report.json
          retention-days: 14
          if-no-files-found: ignore

      - name: Keep a sample on a hidden ref
        if: inputs.keep_sample && steps.run.outputs.video != ''
        run: |
          git config user.name "congress-receipts-bot"
          git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
          v="${{ steps.run.outputs.video }}"; c="${v%.mp4}-cover.jpg"
          vb=$(git hash-object -w "$v"); cb=$(git hash-object -w "$c")
          tree=$(printf '100644 blob %s\treceipt-sample.mp4\n100644 blob %s\treceipt-sample-cover.jpg\n' "$vb" "$c
b" | git mktree)
          commit=$(git commit-tree "$tree" -m "Sample receipt video")
          git push -f -q origin "$commit:refs/previews/receipt-sample"
          echo "::notice title=Sample video::saved to refs/previews/receipt-sample"
```

```
      - name: Our YouTube channel's link (for the site)
        if: steps.run.outputs.mode == 'live'
        continue-on-error: true
        run: npx tsx src/cli.ts channel-info

      - name: Learn from retention
        if: steps.run.outputs.mode == 'live'
        continue-on-error: true
        run: npx tsx src/cli.ts learn

      - name: Build website
        if: always()
        # Public site shows published receipts only. Dry-run test renders never appear on it.
        # TV_CHECK: confirm each receipt's TradingView chart page exists before linking to it.
        env:
          TV_CHECK: "1"
        run: npx tsx src/cli.ts site

      - name: Publish website to Cloudflare Pages
        id: cloudflare
        if: always() && steps.site.outputs.kind == 'cloudflare'
        env:
          CLOUDFLARE_ACCOUNT_ID: ${{ steps.site.outputs.account }}
        run: npx --yes wrangler@4.151.2 pages deploy site/dist --project-name="${{ steps.site.outputs.project }}"
--branch=main --commit-dirty=true

      - name: Publish website to GitHub Pages (until Cloudflare is set)
        id: ghpages
        if: always() && steps.site.outputs.kind == 'github-pages'
        run: |
          cd site/dist
          git init -q -b gh-pages
          git config user.name "congress-receipts-bot"
          git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
          git add -A && git commit -q -m "Site $(date -u +%F)"
          git push -f -q "https://x-access-token:${GH_TOKEN}@github.com/${{ github.repository }}.git" gh-pages
          echo "::notice title=Website::pushed to gh-pages, served at ${{ steps.site.outputs.base_url }}"

      - name: Check the live site answers
        if: always() && (steps.cloudflare.outcome == 'success' || steps.ghpages.outcome == 'success')
        continue-on-error: ${{ steps.ghpages.outcome == 'success' }}
        run: |
          base="${{ steps.site.outputs.base_url }}"
          ok=""
          # Wait up to 3 minutes for the new deploy on the home page, then each other page once.
          tries="1 2 3 4 5 6 7 8 9 10 11 12"
          for page in "" "receipts/" "scoreboard/" "how/" "privacy/" "terms/"; do
            for i in $tries; do
              code=$(curl -s -o /dev/null -w "%{http_code}" "$base/$page") && [ "$code" = "200" ] && break
              sleep 15
            done
            tries="1 2"
            if [ "$code" = "200" ]; then ok="$ok /$page"; else echo "::error title=Live check::$base/$page answere
d $code"; exit 1; fi
          done
          curl -s "$base/" | grep -q "Not financial advice. Facts from public filings." || { echo "::error title=L
ive check::home page is missing the not-advice line"; exit 1; }
          echo "::notice title=Live check::$base answered 200 for:$ok"

      - name: Check the TradingView paid links
        if: always() && (steps.cloudflare.outcome == 'success' || steps.ghpages.outcome == 'success')
        run: |
          base="${{ steps.site.outputs.base_url }}"
          ua="Mozilla/5.0 (compatible; CongressReceipts link check)"
          fail() { echo "::error title=Paid links::$1"; exit 1; }
          # Home and scoreboard: one general link each, labeled, with the disclosure.
          for pg in "home|" "scoreboard|scoreboard/"; do
            name="${pg%%|*}"; path="${pg#*|}"
            h=$(curl -s "$base/$path")
            echo "$h" | grep -q "https://www.tradingview.com/?aff_id=1173299&amp;aff_sub=$name&amp;source=site" ||
fail "$name page has no TradingView link"
            echo "$h" | grep -q '<span class="paid">Paid link</span>' || fail "$name page link is not labeled Paid
link"
```

```
            echo "$h" | grep -q "TradingView pays us" || fail "$name page has no disclosure"
          done
          # Every published receipt page: its chart link, right under the result.
          n=0
          for row in $(node -e 'const l=require("./data/ledger.json");const m=require("fs").existsSync("./data/tra
dingview-symbols.json")?require("./data/tradingview-symbols.json"):{};for(const r of l.receipts.filter(r=>r.status
===="published"))console.log([r.slug,String(r.number).padStart(3,"0"),(m[r.facts.ticker.toUpperCase()]||{}).path|
|"/"].join("|"))' 2>/dev/null); do
            slug="${row%%|*}"; rest="${row#*|}"; num="${rest%%|*}"; tvpath="${rest#*|}"
            h=$(curl -s "$base/receipt/$slug/")
            echo "$h" | grep -q "https://www.tradingview.com${tvpath}?aff_id=1173299&amp;aff_sub=receipt-$num&amp;
source=site" || fail "receipt $num has no chart link"
            echo "$h" | grep -q 'See the full chart on TradingView</a> <span class="paid">Paid link</span>' || fai
l "receipt $num link is not labeled"
            n=$((n+1))
          done
          # The links themselves must open a real TradingView page.
          for u in "https://www.tradingview.com/?aff_id=1173299&aff_sub=home&source=site" $(node -e 'const m=requi
re("fs").existsSync("./data/tradingview-symbols.json")?require("./data/tradingview-symbols.json"):{};for(const v o
f Object.values(m).filter(v=>v.path).slice(-5))console.log("https://www.tradingview.com"+v.path+"?aff_id=117329
9")'); do
            code=$(curl -s -L -A "$ua" -o /dev/null -w "%{http_code}" "$u"); [ "$code" = "200" ] || fail "$u answe
red $code"
          done
          echo "::notice title=Paid links::home and scoreboard labeled and disclosed; $n published receipt pages c
arry their chart link; TradingView links open"

      - name: Check the YouTube authorize link
        if: always() && steps.cloudflare.outcome == 'success' && env.YOUTUBE_CLIENT_ID != ''
        run: |
          base="${{ steps.site.outputs.base_url }}"
          out=$(curl -s -o work/auth.html -w "%{http_code} %{redirect_url}" "$base/auth/youtube")
          code="${out%% *}"; loc="${out#* }"
          if [ "$code" = "302" ] && [[ "$loc" == https://accounts.google.com/o/oauth2/v2/auth\?* ]] && [[ "$loc" =
= *"client_id=$YOUTUBE_CLIENT_ID"* ]] && [[ "$loc" == *"redirect_uri=$(node -p 'encodeURIComponent(process.argv
[1])' "$base/auth/youtube/callback")"* ]]; then
            echo "::notice title=Authorize link::$base/auth/youtube sends you to Google sign-in (accounts.google.c
om) with this channel's client and the callback $base/auth/youtube/callback"
          elif [ "$code" = "200" ] && grep -q "Already connected" work/auth.html; then
            echo "::notice title=Authorize link::already connected; the link says so"
          else
            echo "::error title=Authorize link::$base/auth/youtube answered $code ${loc%%\?*}: $(sed -e 's/<[^>]*
>/ /g' work/auth.html | tr -s ' \n' ' ' | cut -c1-200)"; exit 1
          fi

      - name: Save ledger and parsed filings
        if: always()
        run: |
          git config user.name "congress-receipts-bot"
          git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
          git add data docs || true
          if git diff --cached --quiet; then exit 0; fi
          scripts/secret-scan.sh
          git commit -q -m "Daily run $(date -u +%F)"
          for i in 1 2 3 4; do git pull -q --rebase origin main && git push -q origin HEAD:main && break || sleep
$((2**i)); done

      - name: Health check
        if: always()
        run: |
          set +e
          npx tsx src/cli.ts health
          code=$?
          set -e
          open=$(gh issue list --label health --state open --json number -q '.[0].number')
          if [ "$code" = "2" ]; then
            gh label create health --color B60205 --description "Automated health check" 2>/dev/null || true
            mark="health:$(sha256sum work/health-issue.md | cut -c1-12)"
            printf '\n<!-- %s -->\n' "$mark" >> work/health-issue.md
            if [ -n "$open" ] && gh issue view "$open" --json body,comments -q '.body + ([.comments[].body] | join
(" "))' | grep -q "$mark"; then echo "already reported"
            elif [ -n "$open" ]; then gh issue comment "$open" --body-file work/health-issue.md
            else gh issue create --title "Congress Receipts: something broke" --label health --body-file work/heal
th-issue.md; fi
```

```
          elif [ -n "$open" ]; then
            gh issue close "$open" --comment "Fixed: today's health check passed."
          fi
```